

CSCI-4208: Developing Advanced Web Applications

Week 10: Lecture 18

Express

Express - Introduction

Express is a Node.js Web Framework. Node.js is an amazing tool for building networking services and applications. Express builds on top of its features to provide easy to use functionality that satisfy the needs of Web Servers.

Express is Open Source, free, easy to extend, very performant, and has lots and lots of pre-built packages you can just drop in and use, to perform all kind of server-related tasks.

Express - Installation

Install Express into any project with npm:

- From package.json, add 'express' into the dependencies:
 - ```
"dependencies": {
 "express": "*"
```
- From CLI, in app's root directory:
  - ```
npm install express --save
```

Express - Web Server (Example)

Hello World

We're ready to create our first Express Web Server.

Here is some code:

```
const express = require('express')
const app = express()

app.get('/', (req, res) => res.send('Hello World!'))
app.listen(3000, () => console.log('Server ready'))
```

Save this to an `index.js` file in your project root folder, and start the server using

```
node index.js
```

You can open the browser to port 3000 on localhost and you should see the `Hello World!` message.

Express - HTTP Methods

There is a method for every HTTP verb: `get()` , `post()` , `put()` , `delete()` , `patch()` :

```
app.get( '/', (req, res) => { } )  
app.post('/', (req, res) => { } )  
app.put( '/', (req, res) => { } )  
app.delete('/', (req, res) => { } )  
app.patch( '/', (req, res) => { } )
```

Those methods accept a callback function, which is called when a request is started, and we need to handle it.

Express sends us two objects in this callback, which we called `req` and `res` , that represent the Request and the Response objects.

- Request is the HTTP request object including the request parameters, the headers, the body of the request.
- Response is the HTTP response object that we'll send back to the client.

Express - Request Parameters

Property	Description
.app	holds a reference to the Express app object
.baseUrl	the base path on which the app responds
.body	contains the data submitted in the request body (must be parsed and populated manually before you can access it)
.cookies	contains the cookies sent by the request (needs the <code>cookie-parser</code> middleware)
.hostname	the server hostname
.ip	the server IP
.method	the HTTP method used
.params	the route named parameters
.path	the URL path
.protocol	the request protocol
.query	an object containing all the query strings used in the request
.secure	true if the request is secure (uses HTTPS)
.signedCookies	contains the signed cookies sent by the request (needs the <code>cookie-parser</code> middleware)
.xhr	true if the request is an XMLHttpRequest

Express - Request: GET - data (query string)

Express makes it easy to access a query string with a `Request.query` object.

- This object is filled with a property for each query parameter.
- If there are no query params, it's an empty object.

Demo:

```
const express = require('express');
const app = express();

const handleGET = (request, response) => console.log(request.query);

app.get('/', handleGET )
app.listen(8080);
```

Express - Request: POST - data (json/query string)

If the data was sent as JSON, using Content-Type: application/json, use the `express.json()` middleware:

```
const express = require('express')
const app = express()
app.use(express.json())
```

If the data was sent as query string, using Content-Type: application/x-www-form-urlencoded , use the `express.urlencoded()` middleware:

```
const express = require('express')
const app = express()
app.use(express.urlencoded())
```

In both cases you can access the data by referencing it from `Request.body` :

```
app.post('/form', (req, res) => {
  const name = req.body.name
})
```

Express - Response: Send

How to send a response back to the client using Express

In the Hello World example, the `response.send()` method used to send a simple string as a response, and to close the connection:

```
(request, response) => response.send('Hello World!')
```

- If you pass in a string,
it sets the Content-Type header to: `text/html`
- if you pass in an object or an array,
it sets the Content-Type header to: `application/json`,
& parses that parameter into a JSON.

send() automatically:

- sets the Content-Length HTTP response header.
- closes the connection.

Express - Response: End

Use end() to send an empty response

An alternative way to send the response, without a body is by using the `Response.end()` method:

```
response.end()
```

Express - Response: HTTP Status

Set the HTTP response status

Use the `Response.status()` with `end()`:

```
response.status(404).end()
```

Use the `Response.status()` with `send()`:

```
response.status(404).send('File not found')
```

Express - Response: HTTP Status

Set the HTTP response status

`sendStatus()` is a shortcut:

```
response.sendStatus(200)
// === response.status(200).send('OK')

response.sendStatus(403)
// === response.status(403).send('Forbidden')

response.sendStatus(404)
// === response.status(404).send('Not Found')

response.sendStatus(500)
// === response.status(500).send('Internal Server Error')
```

Express - Response: send JSON

How to serve JSON data using the Node.js Express library

When you listen for connections on a route in Express, the callback function will be invoked on every network call with a Request object instance and a Response object instance.

Example:

```
app.get('/', (request, response) => response.send('Hello World!') )
```

Here we used the `Response.send()` method, which accepts any string.

You can send JSON to the client by using `Response.json()` , a useful method. It accepts an object or array, and converts it to JSON before sending it:

```
response.json({ username: 'Foobar' })
```

Express - Response: Cookies

How to use the `Response.cookie()` method to manipulate your cookies

Use the `Response.cookie()` method to manipulate your cookies.

Examples:

```
response.cookie('username', 'Foobar')
```

This method accepts a third parameter which contains various options:

```
response.cookie('username', 'Foobar', { domain: '.example.org', path: '/administrator', secure: true })
```

```
response.cookie('username', 'Foobar', { expires: new Date(Date.now() + 900000), httpOnly: true})
```

Express - Response: Cookies

The most useful parameters you can set are:

Value	Description
domain	the cookie domain name
expires	set the cookie expiration date. If missing, or 0, the cookie is a session cookie
httpOnly	set the cookie to be accessible only by the web server. See HttpOnly
maxAge	set the expiry time relative to the current time, expressed in milliseconds
path	the cookie path. Defaults to /
secure	Marks the cookie HTTPS only
signed	set the cookie to be signed
sameSite	Value of SameSite

Express - Response: Cookies

A cookie can be cleared with:

```
response.clearCookie('username')
```

Express - Response: HTTP Headers

Access HTTP headers values from a request

You can access all the HTTP headers using the `Request.headers` property:

```
app.get('/', (request, response) => console.log(request.headers) )
```

Use the `Request.header()` method to access one individual request header value:

```
app.get('/', (request, response) => request.header('User-Agent') )
```

Express - Response: HTTP Headers

Change any HTTP header value of a response

You can change any HTTP header value using `Response.set()` :

```
response.set('Content-Type', 'text/html')
```

There is a shortcut for the Content-Type header however:

```
response.type('.html')  
// => 'text/html'  
  
response.type('html')  
// => 'text/html'  
  
response.type('json')  
// => 'application/json'  
  
response.type('application/json')  
// => 'application/json'  
  
response.type('png')  
// => 'image/png'
```

Express - Response: Redirects

How to redirect to other pages server-side

You can create a redirect using the `Response.redirect()` method:

```
response.redirect('/go-there')
```

This creates a 302 redirect. A 301 redirect is made in this way:

```
response.redirect(301, '/go-there')
```

You can specify an absolute path (`/go-there`), an absolute url (`https://anothersite.com`), a relative path (`go-there`) or use the `..` to go back one level:

```
response.redirect('../go-there')  
response.redirect('..')
```

You can also redirect back to the Referer HTTP header value (defaulting to `/` if not set) using

```
response.redirect('back')
```

Express - Routing

Routing is the process of determining what should happen when a URL is called, or also which parts of the application should handle a specific incoming request.

In the Hello World example we used this code:

```
app.get('/', (request, response) => {} )
```

This creates a route that maps accessing the root domain URL / using the HTTP GET method to the response we want to provide.

Express - Routing

Named parameters

Useful for when you want to listen for custom requests,

Example:

Create a REST service that takes a string, and returns its uppercase,
Client can send data with named parameters as part of the URL without needing a query string

```
app.get('/uppercase/:theValue', (request, response) => response.send(request.params.theValue.toUpperCase()) )
```

If we send a request to `/uppercase/test`, we'll get **TEST** in the body of the response.
You can use multiple named parameters in the same URL, and they will all be stored in `req.params`.

Express - Routing

Use a regular expression to match a path

You can use regular expressions to match multiple paths with one statement:

```
app.get(/post/, (req, res) => { } )
```

will match `/post` , `/post/first` , `/thepost` , `/posting/something` , and so on.

Express - CORS

How to allow cross site requests by setting up CORS

A JavaScript application running in the browser can usually only access HTTP resources on the same domain (origin) that serves it.

Loading images or scripts/styles always works, but fetch calls to another server will fail, unless that server implements a way to allow that connection.

This way is called CORS, **Cross-Origin Resource Sharing**.

One very important thing that needs CORS is ES Modules, recently introduced in modern browsers. If you don't set up a CORS policy on the server that allows to serve 3rd part origins, the request will fail.

Express - CORS

Fetch example:

```
> fetch('https://google.com')
```

```
< ▶ Promise {<pending>}
```

```
✖ Failed to load https://google.com/: Redirect from https://google.com/ to https://www.google.it/?gfe\_rd=cr&dcr=0&ei=TidHWtehBcPCXprvpIgF has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource. Origin 'https://flaviocopes.com' is therefore not allowed access. If an opaque response serves your needs, set the request's mode to 'no-cors' to fetch the resource with CORS disabled.
```

```
✖ Uncaught (in promise) TypeError: Failed to fetch https://flaviocopes.com/:1
```

```
> |
```

Express - CORS

A Cross-Origin resource fails if it's:

- to a different domain
- to a different subdomain
- to a different port
- to a different protocol

and it's there for your security, to prevent malicious users to exploit the Web Platform.

But if you control both the server and the client, then you may allow them to talk to each other.

How?

It depends on your server-side stack.

Express - CORS

Example with Express

With Node.js and Express as a framework, use the CORS middleware package.

Here's a simple implementation of an Express Node.js server with CORS:

```
const express = require('express')
const cors = require('cors')
const app = express()

const handleGET = (request, response, next) => response.json({ msg: 'WHOA! with CORS it works!' })

app.get('/', cors(), handleGET )

/* the rest of the app */
```

All you need to do is to import the cors module, and pass it in as a middleware function to an endpoint request handler

Express - Templating

Express is capable of handling server-side **template engines**.

Template engines add data to a view, and generate HTML dynamically.

You can use many different template engines, including:

- Pug,
- Handlebars,
- Mustache,
- EJS,
- and more...

Express - Templating: Pug

To use Pug we must first install it:

```
npm install pug
```

and when initializing the Express app, we need to set it:

```
const express = require('express')  
const app = express()  
app.set('view engine', 'pug')
```

We can now start writing our templates in .pug files.

Create an about view:

```
app.get('/about', (request, response) => response.render('about') )
```

Express - Templating: Pug

and the template in `views/about.pug` :

```
p Hello from Foobar
```

This template will create HTML with the content `<p>Hello from Foobar</p>` .

You can pass a variable into a pug template using

```
app.get('/about', (request, response) => response.render('about', {name: 'Foobar'}) )
```

```
p Hello from #{name}
```

Express - Templating: Pug

This is a short introduction to Pug, in the context of using it with Express.

Look at the Pug guide for more information on how to use Pug.

<https://pugjs.org/api/getting-started.html>

Express - Templating: Handlebars

To use Handlebars we must first install it:

```
npm install hbs
```

Put an `about.hbs` template file in the `views/` folder:

```
Hello from {{name}}
```

Use Express to serve it on `/about` :

```
const express = require('express')
const app = express()
const hbs = require('hbs')

app.set('view engine', 'hbs')
app.set('views', path.join(__dirname, 'views'))

app.get('/about', (request, response) => response.render('about', {name: 'Foobar'}))
app.listen(3000, () => console.log('Server ready'))
```

Express - Templating: React

To render React apps, server-side with express, first install it:

```
npm install express-react-views react react-dom
```

Put an `about.jsx` template file in the `views/` folder:

```
const React = require('react')
class HelloMessage extends React.Component {
  render() {
    return <div>Hello from {this.props.name}</div>
  }
}
module.exports = HelloMessage;
```

Use Express to serve it on `/about` :

```
const express = require('express')
const app = express()
const react = require('express-react-views');

app.set('view engine', 'jsx')
app.engine('jsx', react.createEngine() )

app.get('/about', (request, response) => response.render('about', {name: 'Flavio'}) )
app.listen(3000, () => console.log('Server ready'))
```

Express - Middleware

middleware is a function that hooks into the routing process (*i.e. request-response chain*), to perform an operation.

It's commonly used to edit the request or response objects, or terminate the request before it reaches the route handler code.

It's added to the execution stack like this:

```
app.use( (request, response, next) => {} )
```

This is similar to defining a route, but in addition to the Request and Response objects, there's also a reference to the next middleware function

Always call `next()` at the end of a middleware function, to pass the execution to the next handler, unless you want to prematurely end the response, and send it back to the client.

Express - Middleware

You typically use pre-made middleware, in the form of npm packages.

A big list of the available ones is here: <https://expressjs.com/en/resources/middleware.html>

Express - Middleware

Example 1

One example is `cookie-parser`, which is used to parse the cookies into the `request.cookies` object.

You install it using npm

```
npm install cookie-parser
```

and use it:

```
const express = require('express')
const app = express()
const cookieParser = require('cookie-parser')

app.get('/', (req, res) => res.send('Hello World!') )
app.use(cookieParser())
app.listen(3000, () => console.log('Server ready') )
```

Express - Middleware

Example 2

You can also set a middleware function to run for specific routes only, not for all, by using it as the second parameter of the route definition:

```
const myMiddleware = (request, response, next) => {  
  /* ... */  
  next()  
}  
  
const handleGET = (request, response) => response.send('Hello World!')  
  
app.get('/', myMiddleware, handleGET)
```

Express - Middleware

Store data from Middleware

If you need to store data that's generated in a middleware to pass it down to subsequent middleware functions, or to the request handler, you can use the `Request.locals` object. It will attach that data to the current request:

Example:

Suppose you wanted to store a variable called `name` with the value of `'Foobar'`

```
request.locals.name = 'Foobar';
```

Express - Serving: Static files

How to serve static assets directly from a folder in Express

It's common to have images, CSS and more in a public subfolder, & expose them to the root level:

```
const express = require('express')
const app = express()
app.use(express.static('public'))

/* ... */

app.listen(3000, () => console.log('Server ready'))
```

If you have an `index.html` file in `public/` , that will be served if you now hit the root domain URL (`http://localhost:3000`)

Express - Send files

Express provides a handy method to transfer a file as attachment:

Response.download()

Once a user hits a route that sends a file using this method, browsers will prompt the user for download. The `Response.download()` method allows you to send a file attached to the request, and the browser instead of showing it in the page, it will save it to disk.

```
app.get('/', (req, res) => res.download('./file.pdf'))
```

In the context of an app:

```
const express = require('express')
const app = express()
app.get('/', (req, res) => res.download('./file.pdf'))
app.listen(3000, () => console.log('Server ready'))
```

Express - Send files

You can set the file to be sent with a custom filename:

```
response.download('./file.pdf', 'user-facing-filename.pdf')
```

This method provides a callback function which you can use to execute code once the file has been sent:

```
response.download('./file.pdf', 'user-facing-filename.pdf', (err) => {  
  if (err) {  
    //handle error  
    return  
  } else {  
    //do something  
  }  
})
```

Express - Sessions

How to use sessions to identify users across requests

By default Express requests are sequential and no request can be linked to each other. There is no way to know if this request comes from a client that already performed a request previously.

Users cannot be identified unless using some kind of mechanism that makes it possible.

That's what **sessions** are.

When implemented, every user of your API or website will be assigned a unique session, and this allows you to store the user state.

We'll use the `express-session` module, which is maintained by the Express team.

Express - Sessions

You can install it using npm

```
npm install express-session
```

then import it into your application:

```
const session = require('express-session')
```

This is a middleware, so you install it in Express using:

```
const express = require('express')
const session = require('express-session')
const app = express()
app.use( session('secret': '343ji43j4n3jn4jk3n') )
```

After this is done, all the requests to the app routes are now using sessions. **secret** is the only required parameter, but there are many more you can use. It should be a randomly unique string for your application.

Express - Sessions

The session is attached to the request, so you can access it using req.session here:

```
app.get('/', (req, res, next) => {  
  // req.session  
})
```

This object can be used to get data out of the session, and also to set data:

```
req.session.name = 'Foobar'  
console.log(req.session.name) // 'Foobar'
```

This data is serialized as JSON when stored, so you are safe to use nested objects.

You can use sessions to communicate data to middleware that's executed later, or to retrieve it later on on subsequent requests.

Express - Sessions

Where is the session data stored?

It depends on how you set up the express-session module.

It can store session data in

- memory, not meant for production
- a database like MongoDB
- a memory cache like Redis or Memcached

All solutions store the session id in a cookie, and keep the data server-side. The client will receive the session id in a cookie, and will send it along with every HTTP request.

We'll reference that server-side to associate the session id with the data stored locally.

Memory is the default, it requires no special setup on your part, it's the simplest thing but it's meant only for development purposes.

Express - Sessions

cookie-session is another popular package to manage sessions in express

It stores session data client-side in the cookie.

Since the data is stored in a cookie, its sent back and forth in every single request made by the user. Because of this it's limited in size, as it can only store 4 kilobytes of data.

Cookies also should be secured, but by default they are not.

Express - Validating input

Learn how to validate any data coming in as input in your Express endpoints

Say you have a POST endpoint that accepts the name, email and age parameters:

```
const express = require('express')
const app = express()

app.use(express.json())

app.post('/form', (req, res) => {
  const name = req.body.name
  const email = req.body.email
  const age = req.body.age
})
```

Express - Validating input

How do you server-side validate those results to make sure

- name is a string of at least 3 characters?
- email is a real email?
- age is a number, between 0 and 110?

The best way to handle validating any kind of input coming from outside in Express is by using the `express-validator` package:

Express - Validating input

```
npm install express-validator
```

You require the `check` object from the package:

```
const { check } = require('express-validator/check')
```

We pass an array of `check()` calls as the second argument of the `post()` call. Every `check()` call accepts the parameter name as argument:

```
app.post('/form', [  
  check('name').isLength({ min: 3 }),  
  check('email').isEmail(),  
  check('age').isNumeric()  
], (req, res) => {  
  const name = req.body.name  
  const email = req.body.email  
  const age = req.body.age  
})
```

Express - Validating input

There are many more of these methods, all coming from `validator.js`, including:

```
contains()  
equals()  
isAlpha()  
isAlphanumeric()  
isAscii()  
isBase64()  
isBoolean()  
isCurrency()  
isDecimal()  
isEmpty()  
isFQDN() ,  
isFloat()  
isHash()  
isHexColor()
```

```
isIP()  
isIn() ,  
isInt()  
isJSON()  
isLatLong()  
isLength()  
isLowercase()  
isMobilePhone()  
isNumeric()  
isPostalCode()  
isURL()  
isUppercase()  
isWhitelisted()
```

Express - Validating input

If there is any error, the server automatically sends a response to communicate the error. For example if the email is not valid, this is what will be returned:

```
{
  "errors": [{
    "location": "body",
    "msg": "Invalid value",
    "param": "email"
  }]
}
```

This default error can be overridden for each check you perform, using `withMessage()` :

```
check('name')
  .isAlpha()
  .withMessage('Must be only alphabetical chars')
  .isLength({ min: 10 })
  .withMessage('Must be at least 10 chars long')
```

Express - Validating input

What if you want to write your own special, custom validator? You can use the `custom` validator.

In the callback function you can reject the validation either by throwing an exception, or by returning a rejected promise:

```
app.post('/form', [
  check('name').isLength({ min: 3 }),
  check('email').custom(email => {
    if (alreadyHaveEmail(email)) {
      throw new Error('Email already registered')
    }
  }),
  check('age').isNumeric()
], (req, res) => {
  const name = req.body.name
  const email = req.body.email
  const age = req.body.age
})
```

Express - Validating input

The custom validator:

```
check('email').custom(email => {  
  if (alreadyHaveEmail(email)) {  
    throw new Error('Email already registered')  
  }  
})
```

can be rewritten as

```
check('email').custom(email => {  
  if (alreadyHaveEmail(email)) {  
    return Promise.reject('Email already registered')  
  }  
})
```

Express - Sanitizing input

You've seen how to validate input that comes from the outside world to your Express app.

There's one thing you quickly learn when you run a public-facing server: never trust the input.

Even if you sanitize and make sure that people can't enter weird things using client-side code, you'll still be subject to people using tools (even just the browser devtools) to POST directly to your endpoints.

Or bots trying every possible combination of exploit known to humans.

What you need to do is sanitizing your input.

The `express-validator` package you already use to validate input can also conveniently be used to perform sanitization.

Express - Sanitizing input

Say you have a POST endpoint that accepts the name, email and age parameters:

```
const express = require('express')
const app = express()

app.use(express.json())

app.post('/form', (req, res) => {
  const name  = req.body.name
  const email = req.body.email
  const age   = req.body.age
})
```

Express - Sanitizing input

You might validate it using:

```
const express = require('express')
const app = express()

app.use(express.json())

app.post('/form', [
  check('name').isLength({ min: 3 }),
  check('email').isEmail(),
  check('age').isNumeric()
], (req, res) => {
  const name = req.body.name
  const email = req.body.email
  const age = req.body.age
})
```

Express - Sanitizing input

You can add sanitization by piping the sanitization methods after the validation ones:

```
app.post('/form', [  
  check('name').isLength({ min: 3 }).trim().escape(),  
  check('email').isEmail().normalizeEmail(),  
  check('age').isNumeric().trim().escape()  
], (req, res) => {  
  //...  
})
```

Here I used the methods:

- `trim()` trims characters (whitespace by default) at the beginning and at the end of a string
- `escape()` replaces `<`, `>`, `&`, `'`, `"` and `/` with their corresponding HTML entities
- `normalizeEmail()` canonicalizes an email address. Accepts several options to lowercase email addresses or subaddresses (e.g. `flavio+newsletters@gmail.com`)

Express - Sanitizing input

Other sanitization methods:

- `blacklist()` remove characters that appear in the blacklist
- `whitelist()` remove characters that do not appear in the whitelist
- `unescape()` replaces HTML encoded entities with `<`, `>`, `&`, `'`, `"` and `/`
- `ltrim()` like `trim()`, but only trims characters at the start of the string
- `rtrim()` like `trim()`, but only trims characters at the end of the string
- `stripLow()` remove ASCII control characters, which are normally invisible

Force conversion to a format:

- `toBoolean()` convert the input string to a boolean. Everything except for `'0'`, `'false'` and `"` returns true. In strict mode only `'1'` and `'true'` return true
- `toDate()` convert the input string to a date, or null if the input is not a date
- `toFloat()` convert the input string to a float, or NaN if the input is not a float
- `toInt()` convert the input string to an integer, or NaN if the input is not an integer

Express - Sanitizing input

Like with custom validators, you can create a custom sanitizer.

In the callback function you just return the sanitized value:

```
const sanitizeValue = value => {  
  //sanitize...  
}  
  
app.post('/form', [  
  check('value').customSanitizer(value => {  
    return sanitizeValue(value)  
  }),  
], (req, res) => {  
  const value = req.body.value  
})
```

Express - Handling forms

How to process forms using Express

This is an example of an HTML form:

```
<form method="POST" action="/submit-form">
  <input type="text" name="username" />
  <input type="submit" />
</form>
```

When the user press the submit button, the browser will automatically make a `POST` request to the `/submit-form` URL on the same origin of the page, sending the data it contains, encoded as `application/x-www-form-urlencoded`. In this case, the form data contains the `username` input field value.

Forms can also send data using the `GET` method, but the vast majority of the forms you'll build will use `POST`.

The form data will be sent in the POST request body.

Express - Handling forms

To extract it, you will use the `express.urlencoded()` middleware, provided by Express:

```
const express = require('express')
const app = express()

app.use(express.urlencoded())
```

Now you need to create a `POST` endpoint on the `/submit-form` route, and any data will be available on `Request.body` :

```
app.post('/submit-form', (req, res) => {
  const username = req.body.username
  //...
  res.end()
})
```

Don't forget to validate the data before using it, using `express-validator` .

Express - File uploads in forms

How to manage storing and handling files uploaded via forms, in Express

This is an example of an HTML form that allows a user to upload a file:

```
<form method="POST" action="/submit-form">
  <input type="file" name="document" />
  <input type="submit" />
</form>
```

When the user press the submit button, the browser will automatically make a `POST` request to the `/submit-form` URL on the same origin of the page, sending the data it contains, not encoded as `application/x-www-form-urlencoded` as a normal form, but as `multipart/form-data`.

Server-side, handling multipart data can be tricky and error prone, so we are going to use a utility library called **formidable**. [Here's the GitHub repo](#), it has over 4000 stars and well maintained.

Express - File uploads in forms

You can install it using:

```
npm install formidable
```

Then in your Node.js file, include it:

```
const express = require('express')
const app = express()
const formidable = require('formidable')
```

Now in the `POST` endpoint on the `/submit-form` route, we instantiate a new Formidable form using `formidable.IncomingForm()` :

```
app.post('/submit-form', (req, res) => {
  new formidable.IncomingForm()
})
```

Express - File uploads in forms

After doing so, we need to parse the form. We can do so synchronously by providing a callback, which means all files are processed, and once formidable is done, it makes them available:

```
app.post('/submit-form', (req, res) => {  
  new formidable.IncomingForm().parse(req, (err, fields, files) => {  
    if (err) {  
      console.error('Error', err)  
      throw err  
    }  
    console.log('Fields', fields)  
    console.log('Files', files)  
    files.map(file => {  
      console.log(file)  
    })  
  })  
})
```

Express - File uploads in forms

You'll get one or more `Formidable.File` objects, which give you information about the file uploaded. These are some of the methods you can call:

- `file.size` , the file size in bytes
- `file.path` , the path this file is written to
- `file.name` , the name of the file
- `file.type` , the MIME type of the file

The End.

COMING SOON... TO A MOODLE NEAR YOU!

Labs

Web Server (Express)
Number Guessing Game (REST API)

Homework

Create a REST API